BSc (Hons) Software Engineering - Coventry University
National Institute of Business Management (NIBM)

NB6007CEM - Web API Development

# HTTP Methods, Headers & Status Codes

*Day 4 (AM) - Session 7*

Niranga Dharmaratna - niranga@nibm.lk

## Design Guidelines
## §7 §8 §9

HTTP Methods
Headers
Status Codes
Authentication

# Two Promises, Now Delivered

**Since Session 3:** "We use GET here - Session 7 explains exactly why."
**Since Session 1:** "The device writes. The police read. The API must enforce that line."
**Today:** GET gets its explanation. The enforcement mechanism arrives for the first time.

**Both deferred threads close in this session. This is not new theory - it is the resolution of promises made in Sessions 1, 3, 4, and 5.**

*Design guidelines: §7 HTTP Methods | §8 Headers | §9 Status Codes | §12 Authentication (intro)*

# Session 7 Learning Outcomes

**LO1** Classify HTTP methods by safe and idempotent properties and justify GET as the correct method for all read routes

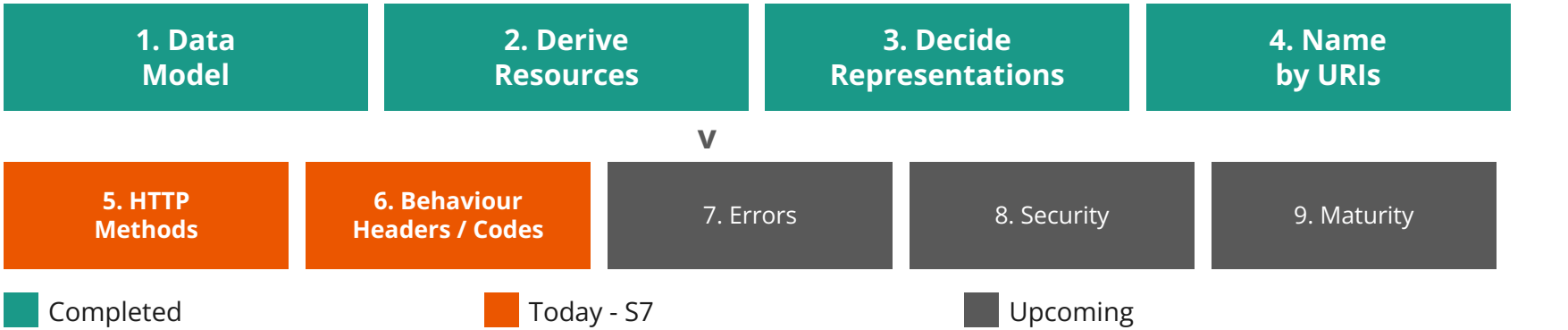
**LO2** Select the correct status code and required response headers for success, client error, and authentication scenarios

**LO3** Read and critique a complete HTTP request/response pair for a device ping against the design guidelines

**LO4** Design the write-read authentication split: API-key for device writes, Basic auth for police reads

**Coursework hook:** API Design (§7, §8, §9) and Security (§12 intro) both advance in this session. The assessed craft is reading generated code and judging whether these contracts are correctly implemented.

# Design Pipeline - Where Session 7 Sits




Steps 5 and 6 are taught together: HTTP methods only mean something when you know what status code and headers accompany them. You cannot specify a route without specifying its full response contract.

# §7 HTTP Methods and Their Properties

# Four Methods, Two Properties

| Method | Safe? | Idempotent? | Key Rule for the Tuk-Tuk API |
|---|---|---|---|
| **GET** | Yes | Yes | All read routes. Retrying a read cannot accidentally create a duplicate ping. |
| **PUT** | No | Yes | Replace the complete record. Calling PUT five times produces the same final state. |
| **DELETE** | No | Yes* | Remove resource.<br>*First call: 200. Subsequent calls: 404 (resource is gone). |
| **POST** | No | No | Create a new resource per call. Not idempotent. Generator mistake: using POST for update. |

**Analogy:**

- ➔ Lift-call button = GET or PUT (press 5 times, lift comes once).
- ➔ Coin-slot turnstile = POST (each press charges separately).
- ➔ Safe/idempotent methods can be retried by any intermediary without side effects; POST cannot.

WHY GET - THE DEFERRED ANSWER FROM SESSIONS 3, 4, AND 5

# GET is safe: a police app retrying a failed read cannot accidentally create a duplicate ping.

*Safe means no side effects on the server. A GET can be cached, prefetched, and retried by any intermediary - browser, proxy, load balancer - without risk. Every read route in this system has used GET since Session 3. Now you know exactly why.*

# Generator Mistakes - HTTP Methods (§7)

**Mistake 1: POST for update operations ("it feels like sending data")**
POST is not idempotent. Calling POST /vehicles/WP-1234 five times with the same body may create five records. PUT replaces the whole resource; calling it five times produces the same final state as calling it once. Generator default: POST. Correct choice: PUT. WSO2 §7.2.

**Mistake 2: GET with a request body to pass filter criteria**
GET is explicitly safe - its semantics do not include a body. Filter criteria belong in the query string (§10.2, Session 9) or in a POST to a processing function resource (§4.5, introduced in Session 5). A request body on a GET violates the safe contract and behaves inconsistently across HTTP clients.

WSO2 §7.2 (PUT), §7.3 (POST) | Assessment dimension: **API Design** | Checkpoint: method choice is explained and justified, not just typed.

# §9 Status Codes: The Protocol Contract

# 2xx Success Codes

**200 OK** Request succeeded. Body contains the result. Correct for GET and for POST where no new resource is created.
**201 Created** New resource created. Location header contains its URI. ETag and Last-Modified should also be returned. Mandatory for any POST that creates a resource.
**202 Accepted** Async processing started; success is not guaranteed. Covered fully in Session 11.

**ANALOGY**

When you register a new SIM card, the counter does not just say "done." They hand you the number you have been assigned - that is where your new identity now lives. A 201 response must work the same way: the Location header tells the client exactly where the created resource can be found. Limit: a SIM number is assigned once and never changes. A LocationPing URI is one of many in a time-series; the resource it points to is the specific ping, not the vehicle.

STATUS CODES ARE CONTRACTS

# 201 + Location - a created resource must tell you where it now lives.

*A client that receives 200 from a POST does not know whether a resource was created, where it is, or whether the request did anything at all. The status code is the server's binding statement about what happened and what the client should do next.*

# 4xx Client Errors - The Request Was the Problem

**400 Bad Request** Malformed, missing fields, values out of range. Generator mistake: returning 200 with an error message in the body.
**401 Unauthorized** Credentials missing or rejected. Response must include WWW-Authenticate. Client may retry with correct credentials.
**403 Forbidden** Credentials valid but resource is off-limits. Client should not retry. See next slide.
**404 Not Found** Resource does not exist. Generator mistake: returning 200 with null or an empty body.
**406 / 415** 406: Accept format not supported. 415: request Content-Type not understood.
**412 Precondition Failed** Conditional request failed (If-Match). Covered in Sessions 9-11.

*5xx codes denote server errors and are generic - no need to document them per endpoint per §9.*

# 401 vs 403 - They Are Not the Same

| 401 Unauthorized | 403 Forbidden |
| --- | --- |
| **"Who are you? I do not recognise these credentials."**<br><br>**Must include:** WWW-Authenticate: Basic realm="tuk-tuk-api"<br><br>**Client action:** Retry with correct credentials.<br><br>**Tuk-tuk example:** X-API-Key header missing from device push. Device can retry once it has the correct key. | **"I know exactly who you are. You are not allowed here."**<br><br>**WWW-Authenticate:** not required. Credentials were valid.<br><br>**Client action:** Do not retry. Permission is denied regardless of credentials.<br><br>**Tuk-tuk example:** Station officer requests another district's vehicles. Identity confirmed; jurisdiction denied. |

**Generator mistake: using 401 and 403 interchangeably. They signal different things - 401 means "retry with credentials"; 403 means "do not retry at all."**

# §8 Headers: The Full Wire Picture

# Request Headers - What the Client Tells the Server

**Accept** Media types the client can handle. Our API supports application/json only. A mismatch triggers 406. Example: Accept: application/json

**Authorization** Credentials for authentication. Format depends on scheme: "Basic {base64(user:pass)}" for police reads; "X-API-Key: {key}" for device writes. Bearer token format belongs to Session 13 (OAuth).

**Content-Type** Format of the request body for PUT and POST. Must be application/json. A mismatch triggers 415. Example: Content-Type: application/json

**Session 8 focus:** Three headers matter for implementation now: Accept, Authorization, Content-Type. Conditional headers (If-Match, If-None-Match, If-Modified-Since) arrive in Sessions 9-11.

# Response Headers - What the Server Tells the Client

**Content-Type** Format of the response body. application/json for all routes. Express sets this automatically via res.json().
**Location** URI of the newly created resource. Mandatory in all 201 responses.
**ETag** Fingerprint of the resource at the moment of response. Returned with 200 and 201. Clients send it back in If-None-Match (caching) or If-Match (concurrency control) - Session 9.
**Last-Modified** Timestamp of most recent modification. Companion to ETag. Returned with 200 and 201.
**WWW-Authenticate** Authentication scheme required. Mandatory in every 401 response. Value: Basic realm="tuk-tuk-api"

**ETag goes in the response today. What clients do with it - If-None-Match, 304 Not Modified - that is Session 9.**

# POST /vehicles/:id/pings - The Wire Contrast

## What Generators Produce

**REQUEST**
POST /vehicles/WP-1234/pings
Content-Type: application/json

{ "lat": 6.9271, "lng": 79.8612 }

**RESPONSE**
HTTP/1.1 200 OK
Content-Type: application/json

{ "success": true }

**Client learns nothing:**
- No URI for the created resource
- No fingerprint (ETag)
- No timestamp

## What §7.3 / §9 Requires

**REQUEST**
POST /vehicles/WP-1234/pings
X-API-Key: dev-WP-1234-secret
Content-Type: application/json

{ "lat": 6.9271, "lng": 79.8612, "speed": 28 }

**RESPONSE**
HTTP/1.1 201 Created
Location: /vehicles/WP-1234/pings/PNG-99999
ETag: "a3f9c2d1"
Last-Modified: Wed, 15 Jan 2025 10:30:00 GMT
Content-Type: application/json

{ "ping_id": "PNG-99999", "vehicle_id": "WP-1234",
"lat": 6.9271, "lng": 79.8612, "speed": 28 }

# §12 intro  First Authentication - Enforcing the Line

# Write Path vs Read Path - Two Clients, Two Schemes

## Write Path - IoT Devices

**Scheme:** API-key (custom header)

**Header:** X-API-Key: dev-WP-1234-secret

Key bound to vehicleId in path. Server validates: key must match vehicle in path. WP-1234's key cannot push a ping for WP-5678.

**Wrong key:** 401

**Valid key:** accept ping, return 201 + Location

## Read Path - Police Users

**Scheme:** HTTP Basic Authentication

**Header:** Authorization: Basic {base64(user:pass)}

Server checks against hardcoded user store for S8 (database not yet in scope).

**Missing or wrong:** 401 + WWW-Authenticate: Basic realm="tuk-tuk-api"

**Valid:** 200 + requested resource

*Admin path (POST/PUT/DELETE /vehicles): Basic auth.*
*All authenticated users have equal access until Session 13 introduces role scopes.*

BASIC AUTH OVER HTTP

# Base64 is not encryption. Basic auth over plain HTTP sends your credentials in cleartext.

*Every intermediate node between client and server can read the Authorization header. HTTPS is what makes any HTTP authentication scheme safe. For this API on Render, HTTPS is provided automatically - but the principle applies everywhere. Analogy: speaking your password aloud in a corridor - everyone nearby hears it. HTTPS is a private room.*

# Complete Route and Status Map for Session 8

| Route | Method | Success | Key Errors | Auth |
|---|---|---|---|---|
| **GET /provinces** | GET | 200 | - | Basic |
| **GET /provinces/:id** | GET | 200, 404 | - | Basic |
| **GET /districts** | GET | 200 | - | Basic |
| **GET /districts/:id** | GET | 200, 404 | - | Basic |
| **GET /stations** | GET | 200 | - | Basic |
| **GET /stations/:id** | GET | 200, 404 | - | Basic |
| **GET /vehicles** | GET | 200 | - | Basic |
| **GET /vehicles/:id** | GET | 200, 404 | - | Basic |
| **GET /vehicles/:id/pings** | GET | 200, 404 | - | Basic |
| **GET /vehicles/:id/last-position** | GET | 200, 404 | - | Basic |
| **POST /vehicles/:id/pings** | **POST** | **201 + Location** | 400, 401, 404 | **API-key** |

**Admin path (Basic auth):** POST /vehicles → 201+Location (400, 401) | PUT /vehicles/:id → 200 (400, 401, 404) | DELETE /vehicles/:id → 200 (401, 404)
LocationPing: append-only (no PUT/DELETE) - GPS history is legal evidence; deletion is a compliance violation. Province, District, PoliceStation: read-only. Role scopes deferred to Session 13.

# Open the Demo

**File: StatusInspector.html**

**Pick a scenario from the left panel:**
- Device pushes a ping (success)
- GET vehicle - not found
- POST ping - missing fields
- GET without Authorization header
- POST with correct API key
- POST with wrong API key
- GET with Accept: application/xml

**Compare: Generator vs §8/§9 required.**

**POST-ping success is the anchor scenario.**

Left column: what a generator produces (200 + { success: true }).

Right column: what §7.3 / §9 requires (201 + Location + ETag + Last-Modified + full body).

**No network calls.**
**Fully offline.**
**All responses simulated.**

# Session 8 Checkpoint - What You Will Build

**Binary pass criteria for the Day 4 checkpoint:**

➔ POST /vehicles/:id/pings returns 201 Created (not 200)
➔ 201 response includes a Location header pointing to the created ping URI
➔ GET /vehicles/:id returns 404 (not 200 with null) when vehicle does not exist
➔ GET without Authorization header returns 401 with WWW-Authenticate header
➔ POST with wrong or missing X-API-Key returns 401

**Every route is now protected. Localhost deployment is not accepted. Commit, redeploy to Render, and share the public URL before the micro-viva.**

*Assessment dimension: API Design (§7, §8, §9) | Deployment and Operation | Functionality*

# Questions & Discussion

*Session 7 - HTTP Methods, Headers and Status Codes*
*Day 4 (AM)*

Next: S8
CRUD + Auth
Implementation

# Disclaimer & Copyrights

The information presented in this presentation is intended for general informational purposes only. It is not meant to be comprehensive or to constitute professional advice. Any reliance you place on such information is strictly at your own risk. While I strive to keep the information accurate and up-to-date, I make no representations or warranties of any kind about the completeness, accuracy, reliability, suitability, or availability of the content.

This presentation may include references or links to third-party websites or content. I do not endorse the views expressed or the information provided on such external sites and am not responsible for the content of any linked site.

Any action you take upon the information presented is strictly at your own discretion. I will not be liable for any losses or damages in connection with the use of this information.

This disclaimer is subject to change without notice.

@ Images and artworks used in this presentation are used for general informational purposes only. It may contain copyrighted material, the use of which may not have been specifically authorised by the copyright owner. This material is available in an effort to explain issues relevant to the topic. This should constitute a 'fair use' of any such copyrighted material.

**Last modified: 27 Jun 2026**